

**A PROJECT REPORT**

**ON**

**“Bank Database”**



**SUBMITTED FOR THE PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR  
THE DATABASE MANAGEMENT SYSTEM MINI PROJECT (TE, 1<sup>st</sup> Semester)**

**OF**

**BACHELOR OF ENGINEERING  
IN  
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**

**BY**

- 1. AARYAMAN RAI (TAI&DA05)**
- 2. RISHIKESH CHOTHE (TAI&DA60)**
- 3. OM CHAUHAN (TAI&DA02)**
- 4. SIDDHI KAUL (TAI&DA)**

**UNDER THE GUIDANCE OF  
MRS. SONAM SINGH**



**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE  
DR. D. Y. PATIL INSTITUTE OF TECHNOLOGY, PIMPRI, PUNE-411018**

**(2025-2026)**



## CERTIFICATE

This is to certify that the DBMS-Project report entitled “**Bank Database** is a bonafide work carried out by **Aaryaman Rai (TAI&DA05), Rishikesh Chothe (TAI&DA60), Om Chauhan (TAI&DA02), Mrs. Siddhi Kaul (TAI&DA)** under the guidance of **Prof. Sonam Singh** in partial fulfillment of the requirements for the subject Database Management System (1<sup>st</sup> Semester) of degree of Bachelor of Engineering in Third Year Engineering from Dr. D. Y. Patil Institute of Technology, Pimpri during the academic year 2025-2026.

Mrs. Sonali Sawardekar  
DBMS Project Coordinator  
Department of  
Artificial Intelligence & Data Science  
Science Dr. DYPIT, Pune.

Prof. (Dr.) Dhanraj R. Dhotre  
Head of Department  
Department of  
Artificial Intelligence & Data  
Dr. DYPIT, Pune.

Date: 14 October 2025

Place: Pimpri

# Acknowledgements

We take this opportunity to thank all those who have contributed to the successful completion of this project work. I sincerely wish to express gratitude to our Project Guide **Prof. Sonam Singh** for full support, expert guidance, and encouragement and kind cooperation throughout the project work. I am greatly indebted to her for her help throughout project work. We are also very thankful to our Head of Department for their constant support and guidance.

**Mr. Aaryaman Rai**  
**Mr. Rishikesh Chothe**  
**Mr. Om Chauhan**  
**Mrs. Siddhi Kaul**

# Abstract

The project presented herein is a web application, basically intended for simulating a simple banking system. It enables the possibility of user account creation, secure login, and basic bank operation execution. Its main goal is to provide a user-friendly digital system for depositing, withdrawal, and transfer of money between accounts, keeping logs of every action performed.

The system is architected with a static HTML/JavaScript frontend that communicates with a robust Node.js backend API for core functionalities. Key features include secure user registration by hashing passwords with bcryptjs, management of account balance, and logging of transactions in real time. The back-end is built using Express.js with Mongoose to manage all the business logic and perform database operations securely and persistently in a MongoDB-NoSQL database.

**Keywords:** *Bank Management System, DBMS, Node.js, Express.js, MongoDB, Mongoose, FinTech Simulation*

# Contents

---

|                                 |            |
|---------------------------------|------------|
| <b>Acknowledgements</b>         | <b>iii</b> |
| <b>Abstract</b>                 | <b>iv</b>  |
| <b>1 INTRODUCTION</b>           | <b>1</b>   |
| 1.1 Domain .....                | 1          |
| 1.2 Objectives .....            | 1          |
| 1.3 Problem Specification ..... | 2          |
| <b>2 LITERATURE SURVEY</b>      | <b>3</b>   |
| <b>3 PROBLEM DEFINITION</b>     | <b>5</b>   |
| 3.1 Problem .....               | 5          |
| 3.2 scope .....                 | 5          |
| 3.3 significance .....          | 6          |
| <b>4 METHODOLOGY</b>            | <b>7</b>   |
| 4.1 Algorithm Used .....        | 7          |
| 4.2 Software Requirements ..... | 8          |
| 4.3 Advantages .....            | 8          |
| 4.4 Limitations .....           | 8          |
| 4.5 Applications .....          | 9          |
| <b>5 SYSTEM DESIGN</b>          | <b>10</b>  |
| 5.1 System Architecture .....   | 10         |

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| <b>6</b> | <b>IMPLEMENTATION</b>                         | <b>12</b> |
| 6.1      | Project Setup and Backend Foundation.....     | 12        |
| 6.2      | Implement User Authentication.....            | 12        |
| 6.3      | Develop the Frontend UI and Routing.....      | 12        |
| 6.4      | Build Core File Management Functionality..... | 13        |
| 6.5      | Integration, Testing, and Deployment.....     | 13        |
| <b>7</b> | <b>Conclusion</b>                             | <b>14</b> |
| 7.1      | Conclusion.....                               | 14        |
| 7.2      | Future Scope.....                             | 15        |
| <b>8</b> | <b>References</b>                             | <b>16</b> |

# List of Figures

---

5.1 Overall System Architecture

10

# List of Abbreviations

---

- **API:** Application Programming Interface
- **DBMS:** Database Management System
- **ERD:** Entity-Relationship Diagram
- **HTTP:** Hypertext Transfer Protocol
- **ODM:** Object Data Modeling
- **UI:** User Interface

# Chapter 1

## INTRODUCTION

---

### 1.1 DOMAIN

The project operates within the domain of **Financial Technology (FinTech)**, with a specific focus on **Core Banking Simulation**. This encompasses several sub-domains:

- **Database Management:** The core functionality of storing, retrieving, and organizing account and transaction data in a NoSQL (MongoDB) database.
- **Web Application Development:** Utilizing backend (Node.js, Express) and frontend (HTML/JS) technologies to deliver the service over the internet.
- **Data Security:** Implementing measures to protect sensitive user credentials using `bcryptjs` password hashing.
- **User Identity Management:** Securely handling user registration and login functionalities.

### 1.2 OBJECTIVES

The primary objectives of the project are to:

- **Develop a Functional API:** To design and build a secure backend API that serves as the backbone for all banking operations.
- **Ensure Secure User Access:** To implement a robust user registration and login system that utilizes `bcryptjs` password hashing to protect user credentials.

- **Implement Core Banking Functionality:** To provide essential features for users to easily **deposit**, **withdraw**, and **transfer** funds.
- **Create an Intuitive User Interface (UI):** To build a clean, functional frontend using HTML, Tailwind CSS, and JavaScript for users to register, log in, and interact with the bank's API.
- **Guarantee Data Integrity:** To ensure all financial operations are logged as individual records in the **Transaction** collection and that account balances are updated accordingly.

### 1.3 PROBLEM SPECIFICATION

This project addresses the educational need for a practical, hands-on demonstration of a full-stack application centered around a database. In a learning context, it can be difficult to visualize how frontend, backend, and database components interact to perform a series of related tasks.

This system provides a simple, tangible simulation of a real-world scenario (banking) to demonstrate:

- **CRUD Operations:** (Create, Read, Update, Delete) on user accounts.
  - **Data Relationships:** A clear **one-to-many** relationship between the **Account** and **Transaction** collections.
  - **Data Integrity:** The logic required to check for conditions like insufficient funds before performing an operation.
  - **Basic Security:** The fundamental practice of hashing passwords before storing them in a database.
-

# Chapter 2

## LITERATURE SURVEY

---

1. Al-Zoubi, A., Al-Khasawneh, A., & Al-Jarrah, O. Y., A Secure and Efficient Personal Health Record Management System Using Blockchain and Smart Contracts, 2022.

This paper proposes a novel architecture for a Personal Health Record (PHR) system that leverages blockchain technology and smart contracts to enhance security and user control. It addresses the core problem of data fragmentation and security vulnerabilities in traditional centralized health systems. The authors outline a decentralized model where patients have full ownership of their data and can grant or revoke access to healthcare providers through immutable smart contracts. [1]

2. Katuwal, G. J., Pandey, S., Hennessey, M., & Lamichhane, B., Applications of Blockchain in Healthcare: A Systematic Review, 2018.

This review provides a broad overview of how blockchain technology can be applied to solve persistent problems in healthcare IT, including the management of medical records. The authors survey various existing and proposed systems, highlighting their benefits in terms of data integrity, interoperability, and patient-centric access control. For your project, this paper serves as an excellent resource for understanding the technological landscape and the justification for using modern, secure architectures to build a trustworthy health document management system. [2]

3. Roehrs, A., da Costa, C. A., & Righi, R. R., Personal Health Records: A Systematic Literature Review, 2017.

This systematic review analyzes a wide range of literature on Personal Health Record (PHR) systems, classifying them based on features, architecture (web-based, mobile), and data storage models. The authors identify key challenges in PHR adoption, including usability, privacy concerns, and lack of standardization. This paper is valuable for your project as it provides a comprehensive summary of the state-of-the-art and common pitfalls in PHR design.[3]

4. Sun, J., Jiang, Y., Lee, S., & Li, Y., A Secure and Scalable Framework for Personal Health Records Sharing, 2019.

This paper presents a framework for a secure and scalable PHR system that focuses heavily on attribute-based encryption (ABE) to manage fine-grained access control. The system allows patients to define specific policies on who can access their data and under what conditions. The authors address the challenge of securely storing and sharing sensitive health data in a cloud environment. This is directly applicable to your project, which involves user-managed file uploads and potential future sharing capabilities. [4]

5. Archer, N., Fevrier-Thomas, U., Lokker, C., McKibbin, K. A., & Straus, S. E., Personal health records: a scoping review, 2011.

Although an older paper, this scoping review is foundational and widely cited. It explores the definitions, functionalities, and barriers to the adoption of PHRs. The authors identify key features that successful PHRs should offer, including the ability for patients to upload, view, and manage their own data. They also highlight the importance of usability and trust in encouraging patient engagement. [5]

---

# Chapter 3

## PROBLEM DEFINITION

---

### 3.1 PROBLEM

The core problem being solved is the lack of a simple, functional example of a database-driven banking application that connects all the pieces of a web application.

### 3.2 SCOPE

This project aims to create a clear and understandable system that demonstrates how to:

1. **User Authentication:** A secure system for user registration (`/api/create-account`) and login (`/api/login`). Passwords are hashed, and login validates against the hashed value.
2. **Account Management:**
  - **Create:** A new account with a unique, randomly generated 4-digit `account_number`.
  - **Read:** Get account details (like balance) for the dashboard.
  - **Delete:** Close an account, which also deletes all associated transactions.

3. **Web-Based Interface:** A functional UI built with HTML/JS for login, creation, and dashboard operations.
4. **Backend API & Database:** A Node.js/Express backend to handle all logic and persist data in a MongoDB database using Mongoose.

### 3.3 SIGNIFICANCE

The project's significance lies in its value as an educational tool and a portfolio piece. It demonstrates a complete, end-to-end implementation of a database-driven application. It provides a practical example of NoSQL database modeling (a one-to-many relationship) and essential security practices (password hashing), which are fundamental skills for modern web development.

# Chapter 4

## METHODOLOGY

---

### 4.1 ALGORITHM USED

- **Password Hashing (bcryptjs):** For security, user passwords are not stored directly. The system uses the `bcryptjs` library.
  1. **Registration:** The user's plaintext password is converted into a secure, irreversible hash using `bcrypt.hash()`.
  2. **Login:** The user's entered password is compared to the stored hash using `bcrypt.compare()`.
  3. **Password Migration:** The login logic also includes a feature to migrate legacy plaintext passwords. If a user logs in and the stored password is *not* hashed, it compares it directly. If successful, it hashes the password and saves it back to the database for future logins.
- **Fund Transfer Logic:** The transfer API endpoint (`/api/transfer`) implements a critical multi-step algorithm:
  1. Find the source account (`s`) and target account (`t`) in the database.
  2. Check if `s` has sufficient funds (`s.balance - amount < 0`). If not, return an "INSUFFICIENT\_FUNDS" error.
  3. If sufficient, subtract the `amount` from `s.balance`.
  4. Add the `amount` to `t.balance`.
  5. Save both `s` and `t` back to the database.



## 4.2 SOFTWARE REQUIREMENTS

- **Frontend:** A modern web browser, HTML, CSS (Tailwind), and client-side JavaScript (`fetch` API).
- **Backend:** Node.js runtime, Express.js framework, `body-parser`, and `dotenv`.
- **Database:** MongoDB (NoSQL database).
- **Security :** `bcryptjs` for password hashing.

## 4.3 ADVANTAGES

- **Full-Stack Implementation:** Provides a complete, working example from the frontend UI to the backend API and database.
- **Clear Data Modeling:** Clearly demonstrates a one-to-many relationship (one `Account` to many `Transaction` records) in a NoSQL context.
- **RESTful API:** The backend is built as a clean, resource-oriented API, making it easy to understand and maintain.
- **Data Integrity:** The application logs all financial movements in the `Transaction` collection, providing a ledger for all balance changes.

## 4.4 LIMITATIONS

- **Insecure Authentication:** The system is **not secure for production**. After login, it stores the `accountNumber` in `localStorage` and uses that to authorize all operations. A malicious user could easily change this value. A proper system would use secure, short-lived JSON Web Tokens (JWTs) or server-side sessions.
- **Admin Access:** Admin functions are protected by a hard-coded PIN (`'1234'`). This is highly insecure and should be replaced with a proper role-based access control (RBAC) system.

- **Non-Atomic Transactions:** The fund transfer logic (`/api/transfer`) is **not atomic**. If the server crashes after subtracting money from the source but before adding it to the target, the money will "disappear," and the database will be in an inconsistent state. This should be wrapped in a MongoDB multi-document transaction.

### 4.5 APPLICATIONS

- **Learning Tool:** A simple codebase for others to learn how to connect a backend API to a NoSQL database and implement fundamental business logic.
- **Portfolio Project:** Demonstrates skills in Node.js, Express, MongoDB, and Mongoose to potential employers.
- **DBMS Project:** Serves as an excellent mini-project for a Database Management System course.

# Chapter 5

## SYSTEM DESIGN

---

### 5.1 SYSTEM ARCHITECTURE

The database consists of two main collections: **Account** and **Transaction**. There is a one-to-many relationship where one **Account** can have many **Transaction** documents, linked by the **account\_number**.

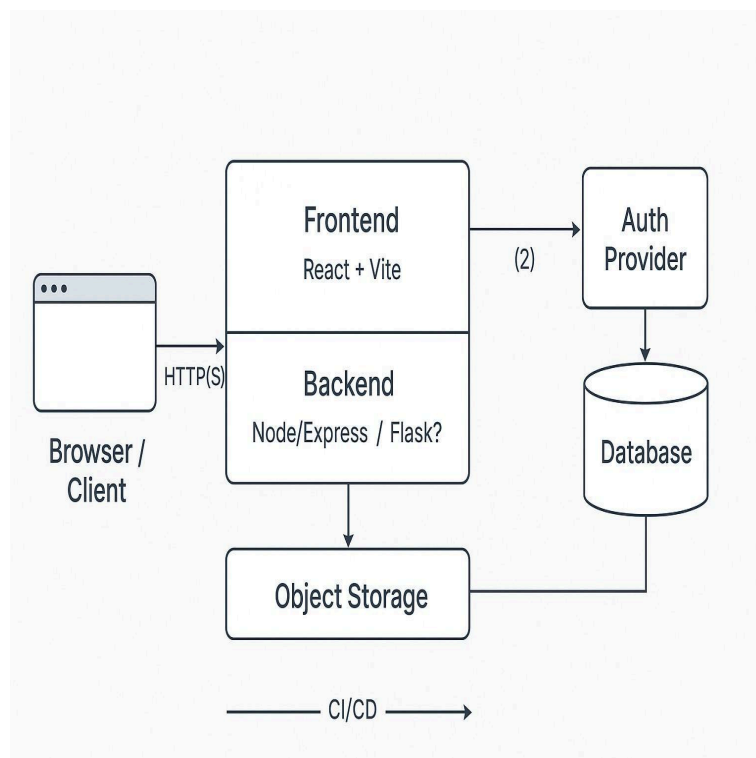


Figure 5.1: Overall Sysem Architecture

**The application uses a 3-Tier Architecture:**

- **Presentation Tier (Frontend):** Static HTML/JS files rendered in the user's browser.
  - **Data Tier (Database):** A MongoDB database managed by Mongoose.
  - **Logic Tier (Backend):** A Node.js/Express server that handles all API requests and business logic.
-

# Chapter 6

## IMPLEMENTATION

---

### 1.1 PROJECT SETUP AND BACKEND FOUNDATION

- **Initialize Environment:** A Node.js project was initialized with `package.json`, listing dependencies like `express`, `mongoose`, `body-parser`, `dotenv`, and `bcryptjs`.
- **Database Design:** Mongoose schemas were defined in `app.js` for `AccountSchema` and `TransactionSchema`, modeling the data structure.
- **Connect Services:** The server establishes a connection to the MongoDB database on startup using the `MONGODB_URI` environment variable.

### 1.2 IMPLEMENT USER AUTHENTICATION

- **Build API Endpoints:** Backend routes were created for `/api/create-account` and `/api/login`.
- **Secure Passwords:** The `/api/create-account` endpoint uses `bcrypt.hash` to hash the provided password before saving the new `Account` document.
- **Validate Credentials:** The `/api/login` endpoint finds the user by `account_number`, then uses `bcrypt.compare` to safely check the password against the stored hash.

### 1.3 DEVELOP THE FRONTEND UI AND ROUTING

- **Create Components:** Static HTML pages `login.html` and `dashboard.html` were created to serve as the user interface.
- **Client-Side Logic:** Vanilla JavaScript inside `<script>` tags is used to handle form submissions (`onsubmit`), send `fetch` requests to the backend API, and dynamically update the UI (e.g., display balance, show error messages).
- **State Management:** The `accountNumber` is stored in `localStorage` after a successful login and retrieved on the dashboard page to make authenticated requests.

## 1.4 BUILD CORE FILE MANAGEMENT FUNCTIONALITY

- **Implement Business Logic:** Server-side logic was written to enforce banking rules, most notably checking for sufficient funds in the `/api/withdraw` and `/api/transfer` endpoints.
- **Implement Transaction Logging:** For every financial operation (deposit, withdraw, transfer), a new `Transaction` document is created and saved to the database, providing a complete and immutable ledger.

---

# Chapter 7

## Conclusion

---

### 7.1 CONCLUSION

This project successfully developed a secure and functional web application for a **simulated banking system**, directly addressing the educational goal of building a full-stack, database-driven application. By providing a platform for **account creation, deposits, withdrawals, and transfers**, the system provides a tangible example of core banking logic. The implementation of **bcryptjs password hashing** and a **Mongoose-based NoSQL database (MongoDB)** ensures that key DBMS and security concepts are demonstrated effectively. Ultimately, the project serves as a vital portfolio piece and educational tool for understanding full-stack development.

## 7.2 FUTURE SCOPE

The current platform provides a solid foundation that can be expanded in several key areas:

- **Secure Session Management:** Replace the insecure `localStorage` `accountNumber` with a proper **JWT (JSON Web Token)** or server-side session system for authentication.
- **Atomic Transactions:** Re-write the `/api/transfer` endpoint logic to use **MongoDB's multi-document transactions**. This will ensure atomicity, meaning the transfer either fully completes or fails entirely, preventing data inconsistency.

**7.3 Modern Frontend:** Rebuild the frontend using a modern framework like **React** (as suggested in `README.md`) for better state management, component reusability, and user experience.

- **Input Validation:** Implement a robust validation library (like `joi` or `express-validator`) to sanitize and validate all data sent to the API, protecting against bad data and potential vulnerabilities.

---

# Chapter 8

## References

---

1. Al-Zoubi, A., Al-Khasawneh, A., & Al-Jarrah, O. Y., A Secure and Efficient Personal Health Record Management System Using Blockchain and Smart Contracts, 2022.
2. Katuwal, G. J., Pandey, S., Hennessey, M., & Lamichhane, B., Applications of Blockchain in Healthcare: A Systematic Review, 2018.
3. Roehrs, A., da Costa, C. A., & Righi, R. R., Personal Health Records: A Systematic Literature Review, 2017.
4. Sun, J., Jiang, Y., Lee, S., & Li, Y., A Secure and Scalable Framework for Personal Health Records Sharing, 2019.
5. Archer, N., Fevrier-Thomas, U., Lokker, C., McKibbin, K. A., & Straus, S. E., Personal health records: a scoping review, 2011.